Advanced Machine Learning

Course Summary

Course Overview

This course delves into crucial aspects of building robust and high-performing machine learning models beyond basic training. It covers essential techniques for evaluating model generalization, improving performance, and handling underfitting and overfitting, strategies for handling data imbalances, the process of optimizing model hyperparameters, and powerful ensemble methods that combine multiple models for superior predictive accuracy and stability. The core objective is to equip learners with the knowledge and tools to develop advanced machine learning solutions for real-world applications.

Week 1: Bagging and Random Forest

I. Ensemble Methods: Combining Models for Better Performance

Ensemble methods combine predictions from multiple separate models (also called base estimators or weak learners) to achieve **better performance and robustness than a single estimator**. The core motivation is the belief that a committee of experts working together is more likely to be accurate than individual experts.

A. Requirements for an Effective Ensemble

For an ensemble to be effective, two key conditions should be met:

- The **base estimators should be as different from each other as possible**.
- The **errors made by each estimator should be different from each other (independent errors)**.

B. Prediction Aggregation

- For **classification problems, the final prediction is the Mode of predictions** from the base estimators.
- For **regression problems, the final prediction is the average of predictions** from the base estimators.

C. Bagging (Bootstrap Aggregating) Bagging is an **ensemble method that builds independent models on each of the random samples and combines their predictions**.

- **Training Process**: **All the weak learners are built in parallel**, meaning they are independent of each other. Each weak learner has **equal weight in the final prediction**.
- **Sampling**: Bagging involves **random sampling with replacement**. Each data point has an equal probability (1/n) of selection at every stage.
- **Why Sampling with Replacement?**:
    - It ensures **independence between samples**, which in turn helps to make base classifiers more **independent/uncorrelated**.
    - This process guarantees that approximately 63% of samples get selected in each iteration. The remaining ~37% are known as "out-of-bag" (OOB) samples and can be used for validation.
    - Sampling without replacement would lead to a decrease in subset size and would require throwing out a lot of data to get reasonable diversity.
- **Benefit**: Bagging helps to **reduce the variance of the model**.
- **Examples**: Bagging Classifier, Random Forest.

D. Random Forest Random Forest is an extension of bagging that **adds an additional random variation into the bagging procedure to create greater diversity amongst the resulting models**.

- **Key Difference from Bagging**: Unlike standard bagging, where all features are considered for splitting a node, in Random Forests, **only a subset of features is selected at random, and the best split feature from this subset is used to split each node in a tree**.
- **Tree Growth Process**:
    - Samples are drawn from the dataset **with replacement** for growing the trees.
    - If M is the total number of input features, a smaller number **m < M features are randomly selected** at each node split.
    - The best split is chosen from these 'm features.
    - Each tree is **grown to the largest extent possible or to a pre-defined depth**.
    - **New data is predicted by aggregating the predictions of all the trees** (mode for classification, average for regression).
- **Hyperparameters (in sklearn)**:
    - n_estimators: The **number of trees in the forest** (default = 100).
    - max_features: The **number of features to consider when looking for the best split**.
    - class_weight: Allows assigning **weights to classes** (e.g., to handle imbalanced data by giving more weight to minority classes).
    - bootstrap: A boolean indicating **whether bootstrap samples are used when building trees** (default = True).

- max_samples: If bootstrap is True, this defines the **number of samples to draw from the dataset to train each base estimator**.
- oob_score: If True, the **out-of-bag (OOB) error is calculated**, which is the average error for each observation using predictions from trees that did not include that observation in their bootstrap sample. This allows for validation during training.

- **Key Functions:**
  - **train _test_split(X, y, test_size=.30, random_state=1, stratify=y):** Splits the data into training and testing sets. The stratify=y parameter ensures that the class distribution is preserved across the splits.
  - **RandomForestClassifier(random_state=1):** Creates a Random Forest classifier with a fixed random seed (random_state=1) for reproducible results.
  - **BaggingClassifier(random_state=1):** an ensemble method that improves model performance by combining multiple estimators trained on random subsets of the data.
  - **GridSearchCV(rf_estimator_tuned, parameters, scoring=acc_scorer,cv=5):** a tuning technique that exhaustively searches over a specified parameter grid to find the best model hyperparameters using cross-validation.

Week 2: Boosting

Ensemble Methods: Sequential Learning (Boosting)

Boosting is an ensemble method that **builds weak learners sequentially**. The core idea is for **successive weak learners to improve the accuracy from the prior learners**.

A. Key Concepts

- **Sequential Training**: Models are built one after another, with each new model attempting to correct the errors of the previous ones.
- **Weighted Learning**: **Weights of each misclassified data point are increased**, and weights for each correctly classified data point are decreased for the subsequent learner. This ensures that **misclassified points from the previous model are given more weight** by the next model.
- **Dependent Samples**: Subsequent samples for training new learners are **dependent on the performance of previous learners**, often emphasizing observations that had higher errors.
- **Final Prediction**: The final prediction is decided by taking the **weighted average or voting** of all the weak learners.

- **Benefit**: Boosting primarily helps to **reduce the bias of the model**.

B. AdaBoost (Adaptive Boosting) AdaBoost is one of the earliest and most popular boosting algorithms.

- **Process**:
    - Initially, it **assigns equal sample weights for each sample**.
    - A weak learner is built on bootstrapped samples according to these weights.
    - After the weak learner is built, AdaBoost **measures its importance based on the errors it made**.
    - **New sample weights are assigned** according to correct and incorrect predictions, and a new bootstrapped dataset is created with probabilities of selection based on these new weights.
    - This process is **repeated for a specified number of times**.
    - The **final prediction is a weighted majority vote/average** of all the weak learners.
- **Hyperparameters (in sklearn)**:
    - base_estimator: The **base estimator from which the boosted ensemble is built**. By default, this is a decision tree with max_depth=1.
    - n_estimators: The **maximum number of estimators (weak learners) at which boosting is terminated** (default = 50).
    - learning_rate: This **shrinks the contribution of each classifier**. There is a trade-off between learning_rate and n_estimators.

C. Gradient Boosting (GBM) Gradient Boosting is another prominent boosting algorithm that differs from AdaBoost in how it handles errors.

- **Key Difference from AdaBoost**: While AdaBoost changes sample weights, **Gradient Boosting fits the next weak learner to the residuals of the previous weak learner**.
- **Process**:
    - A weak learner is built on a subset of the original data.
    - **Errors (residuals) are calculated** after predictions are made by the weak learner.
    - Crucially, **these residuals become the target variable for the new weak learner**. The main aim is to **minimize these residuals**.
    - This process is repeated until there is no significant reduction in residuals or the specified number of estimators is reached.
- **Hyperparameters (in sklearn)**:
    - init: An **estimator object used to compute the initial predictions**. By default, it's a DummyEstimator.
    - learning_rate: **Shrinks the contribution of each tree** (default = 0.1). Similar to AdaBoost, there's a trade-off with n_estimators.

- n_estimators: The **number of boosting stages to perform** (default = 100). Gradient boosting is fairly robust to overfitting, so a large number usually results in better performance.
- subsample: The **fraction of samples to be used for fitting each individual base learner** (default = 1.0).
- max_features: The **number of features to consider when looking for the best split** (default = None).

Ensemble Methods: Advanced Boosting (XGBoost) & Stacking

A. XGBoost (Extreme Gradient Boosting):

XGBoost builds upon the idea of Gradient Boosting with significant modifications to achieve **high computational speed, scalability, and better overall performance**. It is an upgraded implementation of Gradient Boosting.

- **Main Focus**: The primary concentration in XGBoost is **speed enhancement and model performance**.
- **Salient Features for Speed and Scalability**:
  - **Parallelization**: Data is stored in in-memory "blocks." The **optimal split in each tree can be found in parallel** using this block structure, significantly reducing training time.
  - **Cache Optimization**: XGBoost **optimizes the block size for efficient parallelization** to make the best use of hardware.
  - **Out-of-Core Computing**: This feature helps to **handle huge datasets that do not even fit into memory**.
  - **Distributed Computing**: It facilitates **training huge models using multiple similar machines**.
  - **Missing Values Imputation**: XGBoost is **designed to handle missing values internally**. It learns a default direction (left or right child) for missing values during tree construction that optimizes the training loss.
- **Tree Building in XGBoost**:
  - XGBoost sets the initial prediction (e.g., the mean of target values for regression, 0.5 for binary classification).
  - It then calculates residuals.
  - It builds a tree to predict these residuals.
  - A key difference from traditional Gradient Boosting is how it decides the best split: **XGBoost uses 'gain' calculated from a 'similarity score' to find the best split**,

which directly minimizes the loss function, unlike traditional splitting criteria like Gini or entropy.
    - After a tree is built, it calculates the output value for each leaf node.
    - The process repeats until residuals are minimized or the number of estimators is reached.
- **Important Hyperparameters**:
    - learning_rate / eta: Similar to other boosting algorithms, it **shrinks the contribution of each tree** (also called shrinkage). Higher values can lead to less overfitting.
    - gamma: Specifies the **minimum loss reduction required to make a split**. A node is split only if the resulting split gives a positive reduction in the loss function. **Higher gamma values generally lead to lower chances of overfitting**.
    - scale_pos_weight: Controls the **balance of positive and negative weights**, useful for **imbalanced classification problems**.
    - **Column Subsampling (for preventing overfitting)**: XGBoost has hyperparameters to select a subset of columns at various stages:
        - colsample_bytree: Ratio of columns to be used when **constructing each tree**.
        - colsample_bylevel: Ratio of columns for **each new level of a tree**.
        - colsample_bynode: Ratio of columns to be used for **each node (i.e., for each split)**.
        - These three parameters work **cumulatively**.

B. Stacking:

Stacking is an advanced ensemble method that combines **heterogeneous models** using **another "meta-model" for the final prediction**.

- **Concept**: Instead of combining predictions from similar models (like in bagging) or sequentially correcting errors (like in boosting), stacking brings together models of different types (e.g., KNN, Logistic Regression, SVM).
- **Process**: Base models are trained on different folds of the training data. Their predictions on the validation folds are then used as **input features for a final meta-model**, which learns how to best combine these predictions to make the ultimate prediction.
- **Difference from Bagging/Boosting**:
    - **Stacking uses heterogeneous models** and employs a separate meta-model for combining their outputs.
    - In contrast, **bagging typically uses homogeneous models** combined by simple voting or averaging in parallel.
    - **Boosting uses homogeneous models sequentially**, adapting to previous errors and combining via weighted averages.

- Any model can be used to create a stacking model.

- **Key Functions:**
  - train _test_split(X, y, test_size=.30, random_state=1, stratify=y): Splits the data into training and testing sets. The stratify=y parameter ensures that the class distribution is preserved across the splits.
  - AdaBoostClassifier(random_state=1): creates an ensemble model that boosts the performance of weak learners by focusing on previously misclassified instances, with random_state=1 ensuring reproducibility.
  - GradientBoostingClassifier(random_state=1): builds an ensemble of weak learners sequentially, each correcting the errors of the previous one, with random_state=1 ensuring reproducible results.
  - XGBClassifier(random_state=1,eval_metric='logloss'): creates an optimized gradient boosting model using XGBoost, with log loss as the evaluation metric and random_state=1 for reproducibility.
  - GridSearchCV(gbc_tuned, parameters,scoring=acc_scorer, cv=5): a tuning technique that exhaustively searches over a specified parameter grid to find the best model hyperparameters using cross-validation.

Week 3: Model Tuning

I. Cross-Validation: Estimating Model Performance

Cross-validation is a **crucial technique for estimating how well a Machine Learning model is likely to perform in a production environment**. It is especially useful in the absence of large datasets where traditional train-test splits might not adequately represent the universe of data. The primary goal is to achieve a generalised model performance.

A. Concept and Procedure The fundamental procedure of cross-validation involves dividing the available data into multiple segments, or "folds," to simulate real-world unseen data scenarios:

- The data is **divided into k folds**.
- In each iteration, **k-1 folds are used for training** the model, and the kth (remaining) fold is reserved for testing purposes.
- This process is **repeated k times**, ensuring that each fold serves as the test set exactly once.

- Consequently, k different performance scores will be obtained.
- The final model performance is calculated by taking the average of these k scores.
- Additionally, the standard deviation of these scores can be calculated. This allows for the construction of a confidence interval (e.g., with 95% confidence, the model's performance will belong to (average - 2*std. dev.*, *average* + 2std dev)). This provides a better picture of the variance in model performance than a single score.

B. Choosing the Value of 'k': The selection of 'k' is an important decision in cross-validation:

- **k must be an integer**.
- The **minimum value of k has to be 2**, which would involve two iterations.
- The **maximum value of k can be the number of data points** (n). This specific case is known as **Leave One Out Cross Validation (LOOCV)**.
- There is no specific formula to decide the optimal 'k', but k = 10 is usually considered good practice.
- It's important that, regardless of the 'k' value chosen, the resulting training and test data subsets are as representative of the unseen data as possible.
- **Too large a 'k' means less variance across the training sets**, which can limit the differences in models across iterations.
- For a sample size of 'n' and k = p, the **number of records per fold (r) equals n/p**.

II. Model Performance: Underfitting, Overfitting, and Data Leakage

Understanding and addressing common model performance issues is critical for building reliable machine learning models.

A. Underfitting: Underfitting occurs when a model is **too simplistic to capture the underlying patterns in the training data**.

- **Definition**: A model is said to be underfitting when **it is not performing well on the training set**. This means the model is not able to learn from the training set.
- **Reasons for Underfitting**:
    - **Small data size** with a large number of feature**s**.
    - **Less model complexity**.
    - **Irrelevant features** were included in the training.
    - **Imbalanced data** can lead to underfitting.
- **Dealing with Underfitting**:
    - **Increase model complexity**, for example, by moving from linear combinations of features to non-linear combinations.
    - In the case of imbalanced data, use **oversampling or undersampling techniques**.

- For small-sized datasets with many features, use only features that seem important and relevant to the problem.

B. Overfitting Overfitting happens when a model **learns the training data too well, including noise and outliers**, leading to poor generalization on new, unseen data.

- **Definition**: A model is said to be overfitting when it **performs well on training data but not good enough on test/unseen data**. This situation arises when the model starts learning the noise and inaccurate data entries present in the training set.
  - A clear example is Train accuracy = 98.01% while Test accuracy = 55.87%.
- **Reasons for Overfitting**:
  - High model complexity
  - Small dataset
  - Noisy data
- **Detecting Overfitting**:
  - The primary method is to **check for a huge difference in model performance between the train set and the test set**.
  - To confirm overfitting and rule out a biased train-test split, one must check model performance via cross-validation.
- **Dealing with Overfitting**:
  - Regularization techniques
  - Train with more data
  - Remove irrelevant features
  - Decrease model complexity

C. Data Leakage Data leakage is a critical issue where **information from the test set "leaks" into the training process**, leading to an artificially inflated performance on the test data and unreliable evaluation.

- **Definition**: Data leakage occurs when information from the test data is unintentionally used during the model training process. As a result, the model's performance on the test set becomes **unreliable,** as the sanctity of test data is compromised.
- **Causes of Data Leakage**: Data leakage can happen in multiple ways:
  - **Standardizing data** before splitting into training and testing data (e.g., using a z-score).
  - **Imputing missing values** for the entire data before splitting into training and testing data.
  - **Hyperparameter tuning to improve performance on test data**.
- **Prevention**:

- The best way to avoid data leakage is to keep a portion of the sample data away before doing any processing. This held-out portion typically serves as the final test set.
- For transformations where rows influence each other (e.g., z-score), **the testing data should be separately transformed** using the same functions that were used to transform the rest of the data for model building and hyperparameter tuning.
- **One-Hot Encoding** is an exception and can be done before splitting the data, as it does not involve row influence.

III. Handling Imbalanced Data

Imbalanced datasets, where one class significantly outnumbers another, are common in various domains like banking, health, and market analytics. In such datasets, one class is in the majority, and one is in the minority (often less than 5%). Training models on these datasets can cause the model to **give more weightage to the majority class and become biased**.

A. Strategies to Balance Classes: To avoid bias, the class representations need to be balanced. This can be achieved by:

- **Oversampling**: **Increasing the frequency of the minority class**.
  - This technique **creates artificial data points for the minority class**, often by replicating them.
  - **Random oversampling** duplicates random records from the minority class, which is simple but **can cause overfitting**.
- **Undersampling**: **Decreasing the frequency of the majority class**.
  - This technique **removes data points from the majority class**.
  - **Random undersampling** involves removing random records from the majority class, which is simple but **can cause loss of information**.

B. Preference and Advanced Techniques

- In cases of **small data size, oversampling is often preferred** because one "can't afford to lose data points" through undersampling.
- The **Python imblearn module provides more sophisticated resampling techniques**.
  - **SMOTE (Synthetic Minority Oversampling Technique)**:
    - Synthesizes elements for the minority class based on existing ones.
    - It works by **randomly picking a point from the minority class** and computing its k-nearest neighbors.
    - Synthetic points are then added between the chosen point and its neighbors, creating more diverse samples than simple duplication.

- **Tomek links (T-Link)**:
  - These are **pairs of very close instances but of opposite classes**.
  - Removing the instances of the majority class from each Tomek link pair increases the space between the two classes, facilitating the classification process.
- **Cluster centroid based on under-sampling**:
  - This method undersamples the majority class by **replacing a cluster of majority samples with the cluster centroid of a K-Means algorithm**.
  - It keeps a specified number of majority samples by fitting K-Means to the majority class and using the coordinates of the cluster centroids as the new majority samples, seeking to preserve information.

IV. Hyperparameter Tuning: Optimizing Model Performance

Hyperparameters are parameters that govern the entire training process of a machine learning model. Their values are set before the learning process begins, and they have a significant effect on the model's performance.

A. Definition and Benefits

- The process of finding optimal hyperparameters for a model is known as hyperparameter tuning.
- Choosing optimal hyperparameters can lead to improvements in the overall model's performance and can help in reducing both overfitting and underfitting.

B. Process of Hyperparameter Tuning. Fine-tuning hyperparameters typically involves a sequence of steps:

1. Selecting the appropriate model type (e.g., regressor or classifier like sklearn.svm.SVC()).
2. Identifying the corresponding parameter space for the chosen model.
3. Deciding the method for searching or sampling the parameter space.
4. Deciding on the cross-validation scheme to ensure the model will generalize effectively.
5. Deciding on a score function to use to evaluate the model's performance for each hyperparameter combination.

C. Data Split for Tuning

- When tuning hyperparameters, it is essential that the data be split into three parts: **Training, validation, and testing** to prevent data leakage.

V. Hyperparameter Tuning Methods: GridSearchCV & RandomizedSearchCV

Finding optimal hyperparameters can be a **very time-consuming process**, especially for models with a **huge number of hyperparameters**. sklearn provides two common and efficient methods for hyperparameter tuning: GridSearchCV and RandomizedSearchCV.

A. GridSearchCV

- **Concept**: **Grid search is a technique used to find the optimal set of hyperparameters for a model from a provided search space**. It exhaustively considers all parameter combinations specified.
- **Process**: Grid search will **iterate over all possible combinations** in a sequence within the defined search space. For each combination, it trains and tests the model (often using cross-validation) and records the score. Finally, it **gives the best set of hyperparameters based on the best score obtained**.
- **Use Cases**:
    - Best used when there is a **small search space**.
    - Ideal when **time constraints are not an issue**.
- **Limitations**:
    - It **doesn't work well on large search spaces**.
    - It **will find the best set of hyperparameters, but with a high cost** (computationally intensive).

B. RandomizedSearchCV

- **Concept**: Random search is another technique to find the best set of hyperparameters that takes less time than grid search. It **can sample a given number of candidates from a parameter space with a specified distribution**.
- **Process**: Instead of providing a discrete set of values, a **statistical distribution** can be provided for each hyperparameter. Values for different hyperparameters are picked at random from this combined distribution. A fixed number of parameter settings (given by n_iter) are sampled from these distributions; not all combinations are evaluated.
- **Motivation**: The motivation to use random search over grid search is that for many cases, hyperparameters are not equally important. Research suggests that for most datasets, only a few hyperparameters truly matter, making grid search a poor choice for new datasets.
- **Use Cases**:
    - Best used when there is a large search space.
    - Preferred when time constraints are present.
- **Benefits**:
    - Randomized search is known to give better results as compared to grid search.
    - It has a higher chance of hitting the right combination than grid search.
    - It allows passing a range instead of just discrete numbers.

C. Key functions

- **GridSearchCV:** a tuning technique that exhaustively searches over a specified parameter grid to find the best model hyperparameters using cross-validation.

- **RandomizedSearchCV:** a hyperparameter tuning technique that searches over a random subset of parameters to find the best model configuration using cross-validation, offering a faster alternative to GridSearchCV.